

Q. How does Cloud Foundry compare with Docker and Kubernetes?

A. Docker and Kubernetes emerged independently, between 2010 and 2020, each tackling similar challenges. Each solution gathered its own community, with some gaining more traction over time, leading to greater adoption. Container technology itself has been available in Linux for a long time, but it wasn't widely adopted until Docker enhanced the developer experience around it.

Docker packages source code into a container based on the specifications provided, resulting in a container image or an immutable artefact that can be deployed on any compute resource, like a VM. Docker attempted container orchestration with Docker Swarm, which eventually fell out of use as Kubernetes became the preferred orchestrator. Kubernetes offers ways to handle networking and ensure that containers run consistently. It provides the capability to maintain, say, five instances of a container, making sure they're always available. Though Kubernetes doesn't necessarily simplify things, it does offer comprehensive container orchestration.

Cloud Foundry also handles source code to immutable artefact conversion using Linux-based containment (LXC). It can manage containers to ensure they're always available, support multiple instances of applications, and streamline networking, logging, and orchestration within its ecosystem. However, Cloud Foundry has not gained the same popularity as Docker or Kubernetes.

Q. Are there any instances where Cloud Foundry is integrating with Kubernetes or Docker?

A. Yes, for two reasons: first, the popularity of certain projects tends to steer the community in a specific direction. Second, after

creating a Docker container, there are numerous deployment options, but if a standard, compliant, and secure deployment path existed, many companies would prefer it.

In isolation, Kubernetes is limited as it provides only a few core functions; to make it genuinely useful, it requires a complete ecosystem, which is what the CNCF (Cloud Native Computing Foundation) ecosystem offers. This includes a build service, deployment tools, CI/CD pipelines, logging, monitoring, and security tools, all of which collectively make Kubernetes functional. Kubernetes deployment complexity also grows with team structures, as managing consistency and updates across clusters can quickly become complicated; using Kubernetes alone can feel cumbersome. Personally, I believe developers aren't supposed to be wasting their time with Kubernetes.

Cloud Foundry provides a higher-level abstraction over Kubernetes, with constructs like 'orgs' and 'spaces', mapping them to Kubernetes components, simplifying the experience. Developers can use Cloud Foundry's own tools or integrate other cloud-native options like Istio or Envoy for networking.

Q. You made a very interesting statement that "developers are wasting their time with Kubernetes." Could you elaborate?

A. Infrastructure abstractions like Kubernetes are meant to be the "boring" part of the stack eventually. So, when I say developers sometimes waste time, I mean developers don't always need the trendiest solution but a reliable one, where core challenges are already solved. This lets them focus on what truly matters instead of making the basics of their workloads function.

There's room for both established and emerging projects in a developer's toolkit, and developers need to find a middle ground for themselves.

Trends come and go, and very few technologies including Cloud Foundry have withstood the Docker and the Kubernetes wave, and beyond. It's actively developed and continues to serve infrastructure teams who are now focused on optimising AI workflows. Cloud Foundry's strength lies in its community of infrastructure enthusiasts who seek a platform that provides first-class multi-tenant support over any infrastructure, with a straightforward approach to services, workloads, and networking. It's designed to handle various workloads, from tomorrow's AI to today's cloud-native apps, containers, and beyond.

Developers often focus on a technology until it becomes "boring," at which point they move on to higher order problems and solution sets. But Cloud Foundry has seen its peak, held huge events, gained a large following, and remains in use at major companies. For example, JPMorgan runs thousands of containers in production with Cloud Foundry, as do some of SAP's clients.

Q. How does this foundation support scalable application deployment across multiple clouds?

A. Cloud Foundry itself was always designed with the goal of abstracting underlying cloud infrastructure from all the workflows layered above it. Envisioned as a PaaS, Cloud Foundry was created to operate on any hyperscaler or cloud provider you install it on. It is extremely scalable across single or multiple clouds. Cloud Foundry BOSH was inspired by Google's BORG (often cited as a precursor to Kubernetes).

BOSH was designed as a next-generation BOSH, with 'S' following 'R' and 'H' after 'G', symbolising this progression. Cloud Foundry was envisioned to operate at hyperscale, efficiently handling hundreds of thousands of containers. BOSH, or the BOSH Director, serves as a